Week 10: Objects, transactions and security

Learning outcomes: CLO1, CLO4

Lesson

A **view** is a saved query that you can use like a table. An **index** helps MySQL find rows faster, but it takes space and slows down writes. Use `EXPLAIN` to see whether a query uses an index; on a small table the difference may be tiny. A **stored procedure** is a named block of SQL that you run with CALL, and a **trigger** runs automatically when rows are inserted, updated or deleted. Create all of these in `student_work`.

A **transaction** lets you commit related changes together or roll them back together. If a statement fails inside a transaction, MySQL usually cancels only that statement and the transaction stays open, so check what happened and run ROLLBACK if the whole operation should be cancelled. (A deadlock is an exception: MySQL then rolls back the whole transaction.) **ACID** stands for atomicity, consistency, isolation and durability. Remember that CREATE and other DDL statements commit automatically.

Give each account only the permissions it needs (**least privilege**). Your `practice` account can only read `cambodia`, but it can do anything in `student_work`.

The **three-level architecture** separates the external level (what each user or program sees, such as a view), the conceptual level (all tables and relationships) and the internal level (how the data is stored). **Physical data independence** means the storage can change, for example by adding an index, without changing the tables. **Logical data independence** means the tables can change while views keep showing users the same columns, so their programs keep working.

Worked example

Run `example.sql` (see Run a SQL file (`setup/README.md`)) and read the output. Then open `work.sql`.

Tables for this week

Read from the `cambodia` database and make your changes in `student_work`. PK means primary key and FK means foreign key. Every column is `NOT NULL`. The sample rows are the first two rows of each table, sorted by code.

`cambodia.province`

Column	Type	Key	Meaning
`province_pcode`	`CHAR(4)`	PK	Province code
`province_name`	`VARCHAR(100)`		Province name (English)
`area_km2`	`DECIMAL(12,1)`		Area in km², one decimal place

Sample rows:

province_pcode	province_name	area_km2
KH01	Banteay Meanchey	6145.9
KH02	Battambang	11869.0

`cambodia.district`

Column	Type	Key	Meaning
`district_pcode`	`CHAR(6)`	PK	District code
`district_name`	`VARCHAR(100)`		District name (English)
`area_km2`	`DECIMAL(12,1)`		Area in km², one decimal place
`province_pcode`	`CHAR(4)`	FK	Province code

Sample rows:

district_pcode	district_name	area_km2	province_pcode
KH0102	Mongkol Borei	807.2	KH01
KH0103	Phnum Srok	761.7	KH01

Join keys: `district.province_pcode = province.province_pcode`. Join on the codes, not the names.

Objects that `example.sql` creates in `student_work`:

Object	Columns or parameters
`w10_province` (table)	Same columns as `cambodia.province`
`w10_audit` (table)	`province_pcode CHAR(4)`, `deleted_at TIMESTAMP(6)` (time of the delete)
`w10_district_counts` (view)	`province_pcode`, `district_count`
`w10_districts` (procedure)	Input `code CHAR(4)`; returns `district_pcode`, `district_name`

For the index task, create your own copy of the district table. These objects exist only after you run the CREATE statements.

Run `DESCRIBE cambodia.province;` to see a table definition in MySQL. The data guide (`data/README.md`) explains where the data comes from.

Lab tasks

1. Create a view in `student_work` that shows the number of districts in each province.
2. Copy the district rows with `CREATE TABLE student_work.w10_district AS SELECT * FROM cambodia.district;` (this copies the data but not the keys or indexes). Check `SHOW INDEX FROM student_work.w10_district;`, run EXPLAIN on a query that filters by `province_pcode`, add an index on that column, and run EXPLAIN again. With only 197 rows, the time difference will be very small; compare the plans.
3. Open two SQL sessions and run `START TRANSACTION;` in both. In session A, update one row of your copy and do not commit. In session B, update the same row: it waits for A's lock. Roll back A, let B finish, then roll back B. Describe what happened.
4. Create a procedure that lists the districts of the province code it receives, and call it with a code from the data.
5. Create an AFTER DELETE trigger on your province copy that writes to an audit table. Delete one row inside a transaction, check the audit table, then roll back.
6. Self-study: run `SHOW GRANTS;` for the practice account. Then write (on paper, without running them) the CREATE USER, GRANT and REVOKE statements for an app account that may only SELECT from `cambodia`. The practice account is not allowed to create users or grant privileges.

Homework

1. Use the ACID properties to explain what you saw in tasks 3 and 5.
2. Submit your view, index, procedure and trigger scripts with their output, plus your written GRANT and REVOKE statements. Note that the audit row is rolled back too, because it was written in the same transaction.
3. Use your view to explain the three-level architecture and data independence.

Capstone sprint

C10: Add the views, indexes, procedures or triggers your project needs, and describe the permissions your app account should have.

Submit

Write your answers in `work.sql` and label each one, for example `-- Lab 3`. Answer the questions in `reflection.md` and save your query output in `results/`. Then commit and push to your repository. See SUBMISSIONS.md for details.